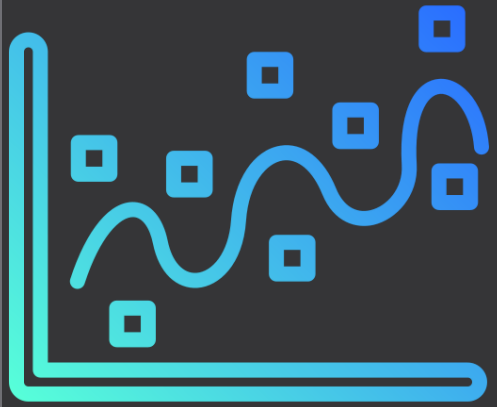


Regularization
The Problem of Overfitting
Lecture 14 – HCCDA-AI

Imran Nawar

Slides adapted from Andrew Ng, Machine Learning (Stanford University / Coursera)

31 Oct 2026

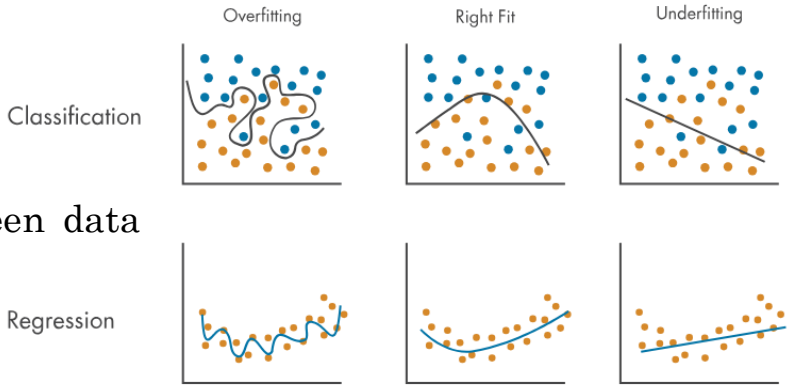
Regularization: The Problem of Overfitting

Overfitting occurs when a model performs exceptionally well on the training data but poorly on new, unseen data.



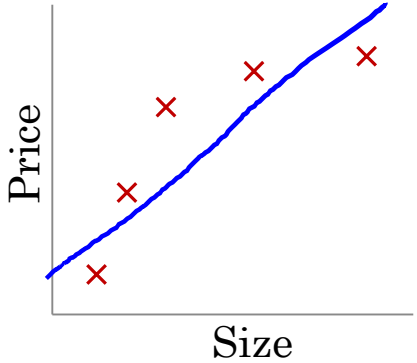
Model Fitting Scenarios:

- **Overfitting:**
  - The model learns noise and specific details of the training set, leading to poor generalization.
- **Just Right (Good Generalization):**
  - The model captures patterns well and generalizes effectively to new data.
- **Underfitting:**
  - The model does not capture patterns well, leading to poor performance on both training and validation data.



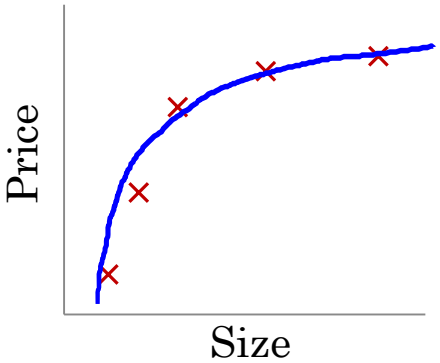
Goal: Build a model that generalizes well to unseen data while avoiding overfitting.

Example: Linear regression (housing prices)



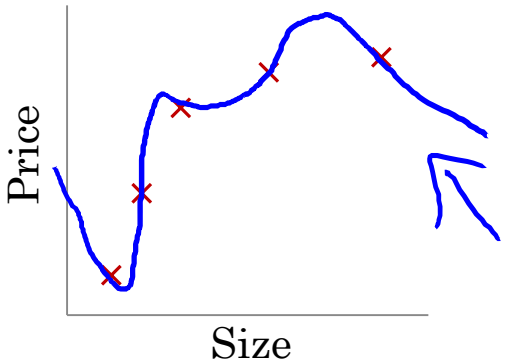
$\theta_0 + \theta_1 x$

"Underfit" "High bias"



$\theta_0 + \theta_1 x + \theta_2 x^2$

"Just right"

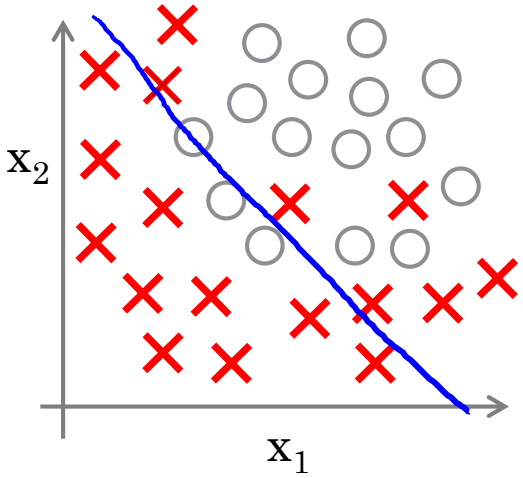


$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$

"Overfit" "High variance"

Overfitting: If we have too many features, the learned hypothesis may fit the training set very well ($J(\theta) = \frac{1}{2m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)})^2 \approx 0$), but fail to generalize to new examples (predict prices on new examples).

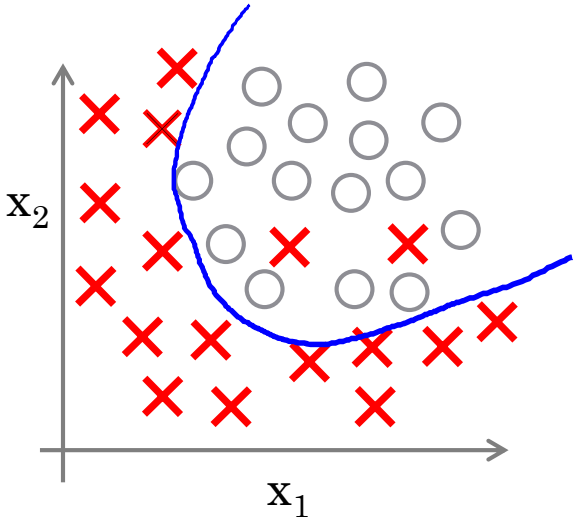
Example: Logistic regression



$h_\theta(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$

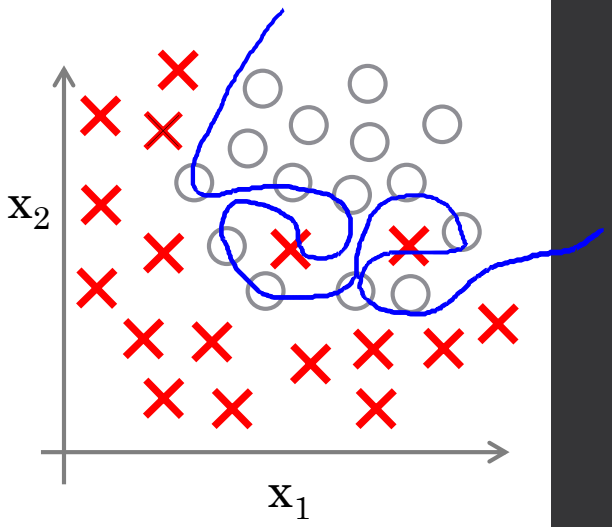
(g = sigmoid function)

"Underfit"



$g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2 + \theta_5 x_1 x_2)$

"Just right"

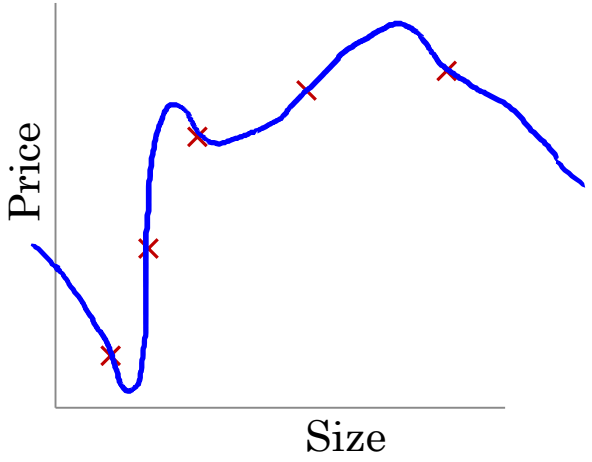


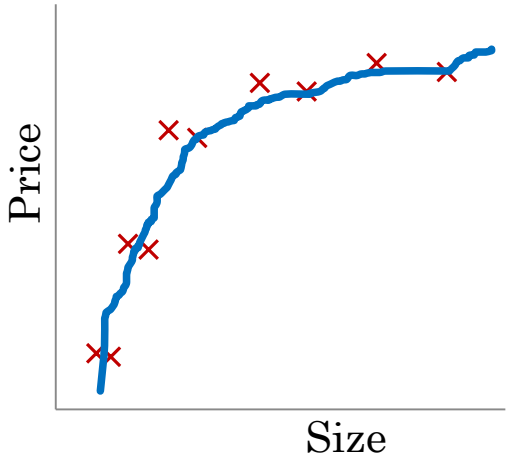
$g(\theta_0 + \theta_1 x_1 + \theta_2 x_1^2 + \theta_3 x_1^2 x_2 + \theta_4 x_1^2 x_2^2 + \theta_5 x_1^2 x_2^3 + \theta_6 x_1^3 x_2 + \dots)$

"Overfit"

Addressing Overfitting:

1. The number one tool to use against overfitting is to get more training data.
   - **Increase Training Data:** More diverse and representative data helps the model generalize better and reduces overfitting.

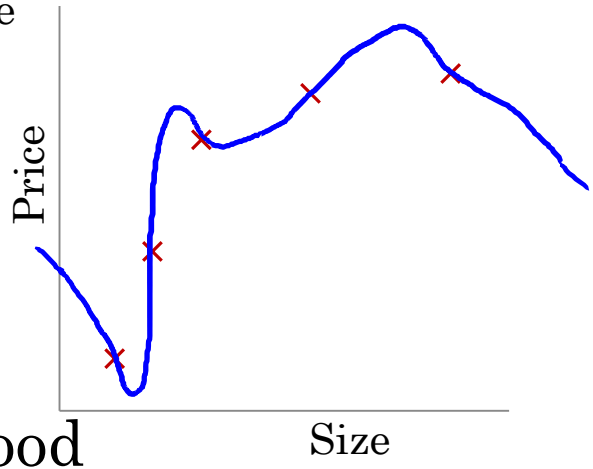




Addressing Overfitting:

2. A second option for addressing overfitting is to see if you can use fewer features.

$x_1 =$ size of house
$x_2 =$ no. of bedrooms
$x_3 =$ no. of floors
$x_4 =$ age of house
$x_5 =$ average income in neighborhood
$x_6 =$ kitchen size
$\vdots$
x_{100}



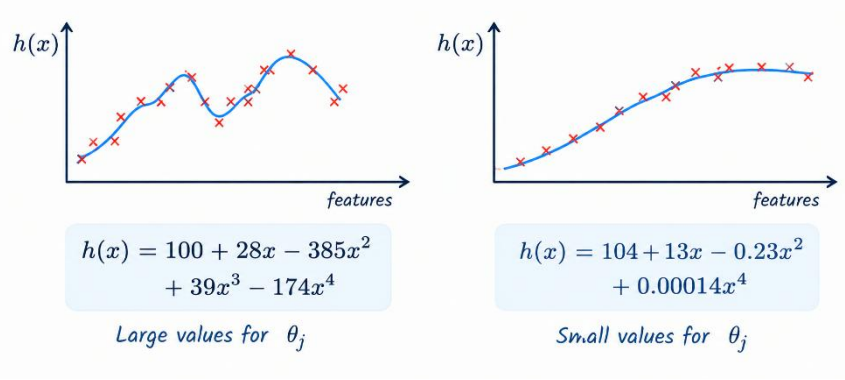
- **Reduce number of features.**
  - Manually select which features to keep.
  - Model selection algorithm.

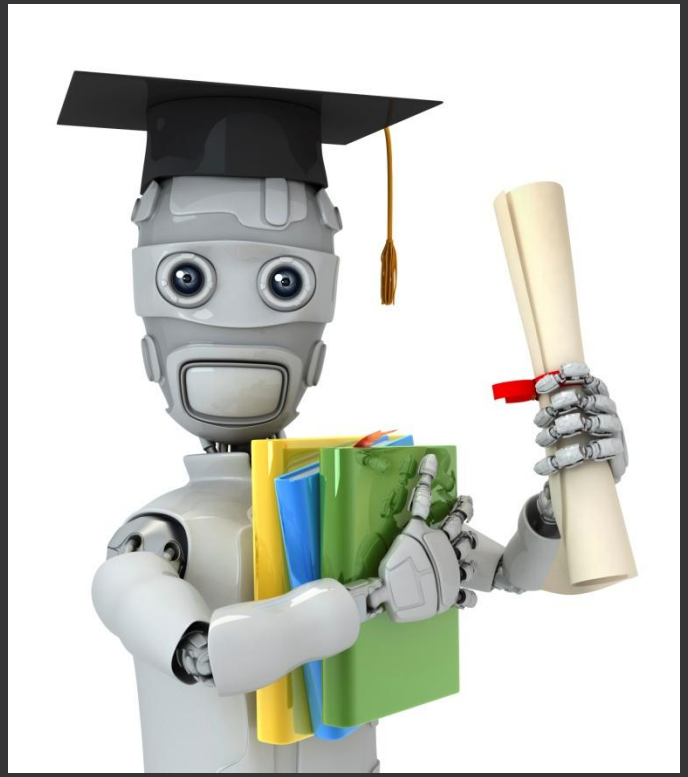
Addressing Overfitting:

3. The third option to reduce overfitting is to use regularization.

Regularization.

- Helps minimize overfitting and improves the generalization of the learning algorithm.
- Keeps all features but reduces the magnitude of model parameters θ_j (coefficients) to prevent excessive reliance on any single feature.
- Works well when there are many features, each contributing partially to predictions y.
- Provides a controlled way to reduce feature impact without completely eliminating them, unlike feature selection.



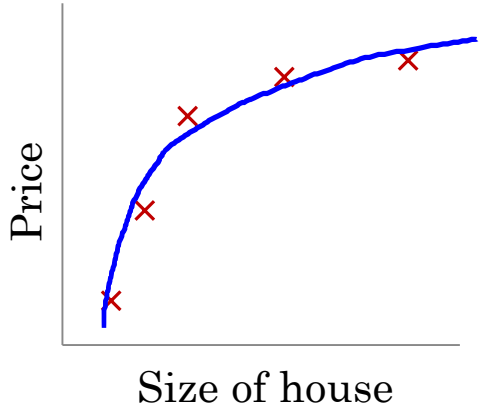


Machine Learning

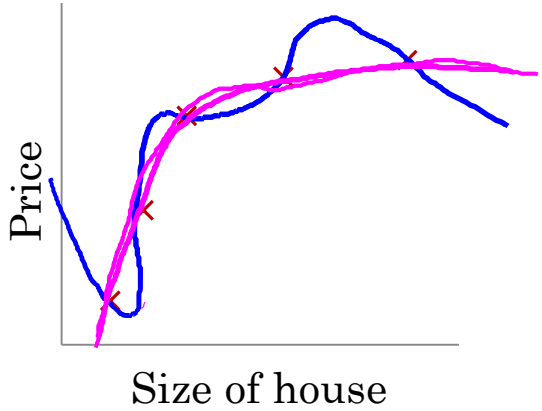
Regularization

Cost Function

Intuition



$\theta_0 + \theta_1 x + \theta_2 x^2$



$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$

Suppose we penalize and make θ_3, θ_4 really small.

$$\min_\theta \frac{1}{2m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)})^2 + 1000\,\theta_3^2 + 1000\,\theta_4^2$$

$\theta_3 \approx 0 \qquad \theta_4 \approx 0$

Regularization.

Small values for parameters $\theta_1, \ldots, \theta_n$

- "Simpler" hypothesis
- Less prone to overfitting

Housing:

- Features: $x_1, x_2, \ldots, x_{100}$
- Parameters: $\theta_0, \theta_1, \theta_2, \ldots, \theta_{100}$

$$J(\theta) = \frac{1}{2m}\left[\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^{n} \theta_j^2\right]$$

$\theta_1, \theta_2, \theta_3, \ldots, \theta_{100}$

Regularization.

$$J(\theta) = \frac{1}{2m}\left[\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2 + \lambda\sum_{j=1}^{n}\theta_j^2\right]$$

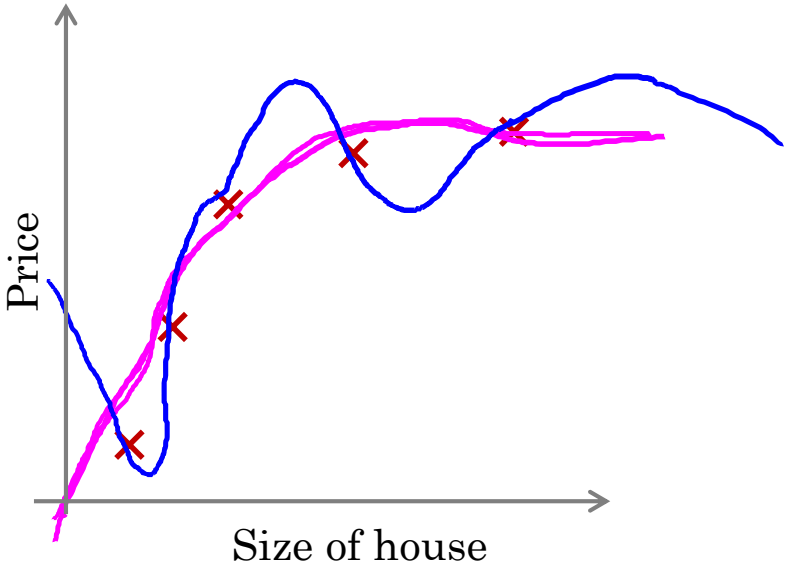
$$\min_\theta J(\theta)$$

$\lambda\sum_{j=1}^{n}\theta_j^2$ ⟶ Regularization term

- Add this regularization term at the end to shrink every single parameter, $\theta_1, \theta_2, \theta_3$, ...

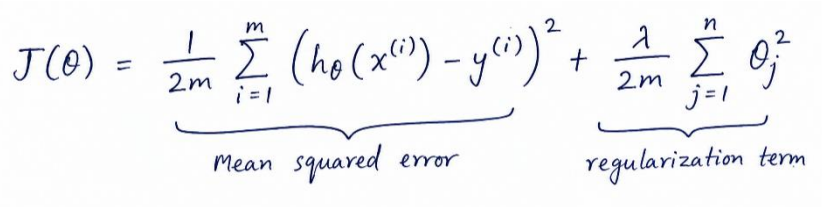
λ ⟶ "Lambda" Regularization parameter.

- λ is divided by 2m so that both the 1st and 2nd terms are scaled by $\frac{1}{2m}$. Scaling both terms the same way makes it easier to choose a good value of λ.
- By convention we are not going to penalize the parameter θ_0,



Regularization.

$$J(\theta) = \frac{1}{2m}\left[\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^{n}\theta_j^2\right]$$



This new cost function trades off two goals:

- **1st:** Fitting the training data well.
- **2nd:** Keeping the parameters small.

Impact of λ in Regularization

- **If $\lambda = 0$:** it means we are not using regularization term.
- **If *λ is too big* (*enormous*):** under fit.
  - The model applies excessive regularization, making it too simple and leading to underfitting.
- **If λ is in between:** Balance "just right"

In regularized linear regression, we choose θ to minimize

$$J(\theta) = \frac{1}{2m}\left[\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2 + \lambda\sum_{j=1}^{n}\theta_j^2\right]$$

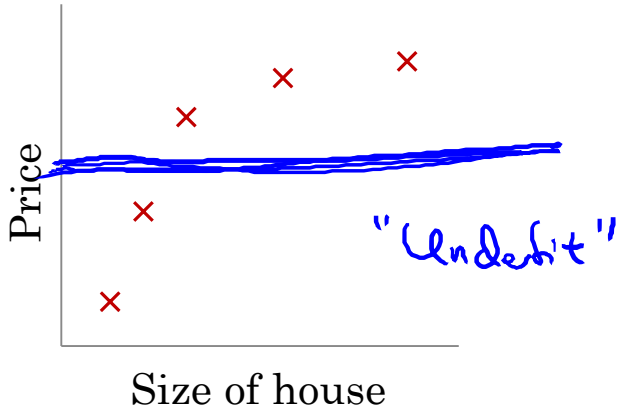
What if λ is set to an extremely large value (perhaps too large for our problem, say $\lambda = 10^{10}$)?

1. Algorithm works fine; setting λ to be very large can't hurt it.
2. Algorithm fails to eliminate overfitting.
3. Algorithm results in underfitting. (Fails to fit even training data well).
4. Gradient descent will fail to converge.

In regularized linear regression, we choose θ to minimize

$$J(\theta) = \frac{1}{2m}\left[\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2 + \lambda\sum_{j=1}^{n}\theta_j^2\right]$$

What if λ is set to an extremely large value (perhaps for too large for our problem, say $\lambda = 10^{10}$)?



$h_\theta(x)$

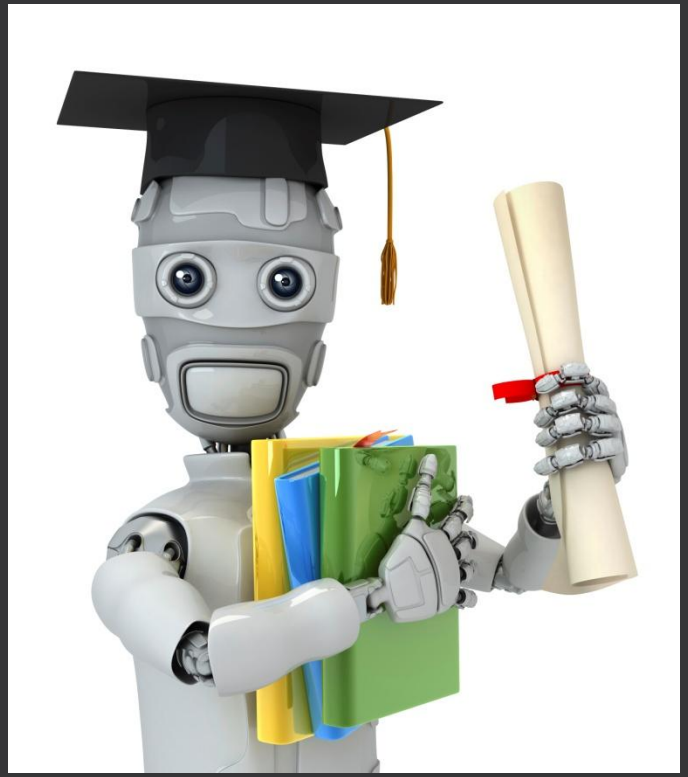
$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$

$\theta_1, \theta_2, \theta_3, \theta_4$

$\theta_1 \approx 0, \theta_2 \approx 0$

$\theta_3 \approx 0, \theta_4 \approx 0$

$h_\theta(x) = \theta_0$



Machine Learning

Regularization

Regularized Linear Regression

Regularized linear regression

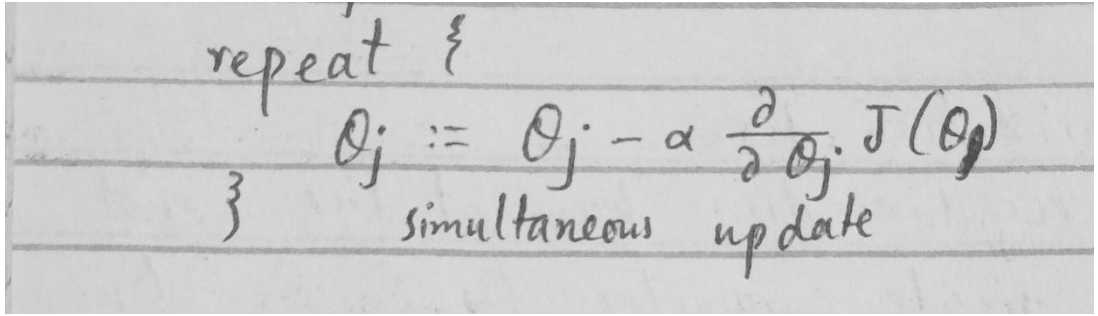
Cost Function: (regularized)

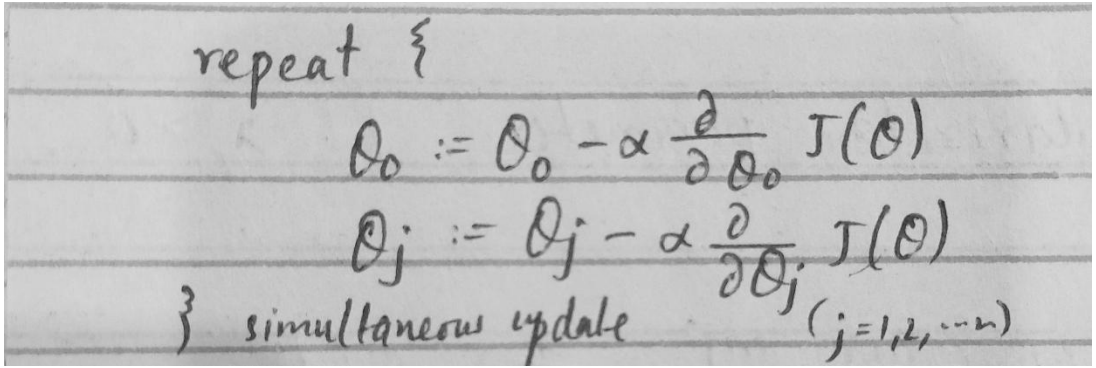
$$J(\theta) = \frac{1}{2m}\left[\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2 + \lambda\sum_{j=1}^{n}\theta_j^2\right]$$

$$\min_\theta J(\theta)$$

Regularized linear regression

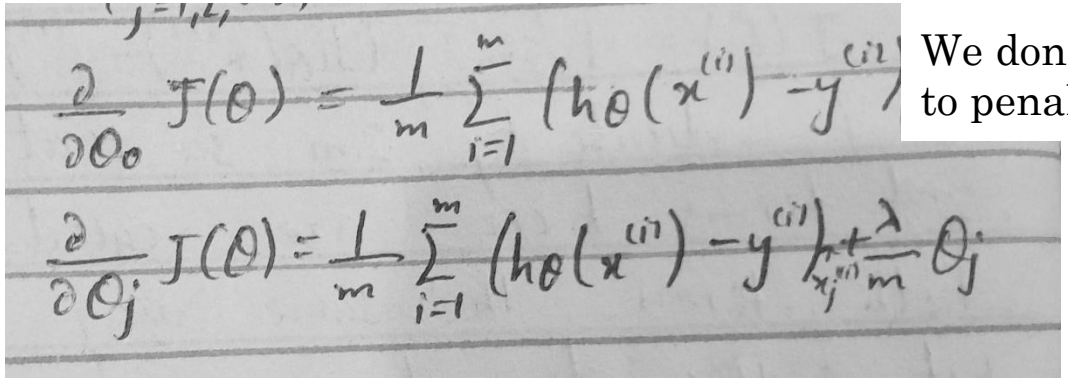
Gradient Descent:



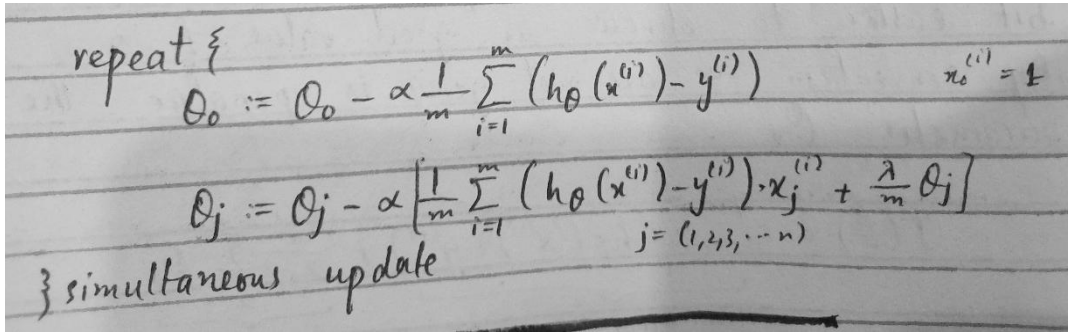


Regularized linear regression

Gradient Descent:

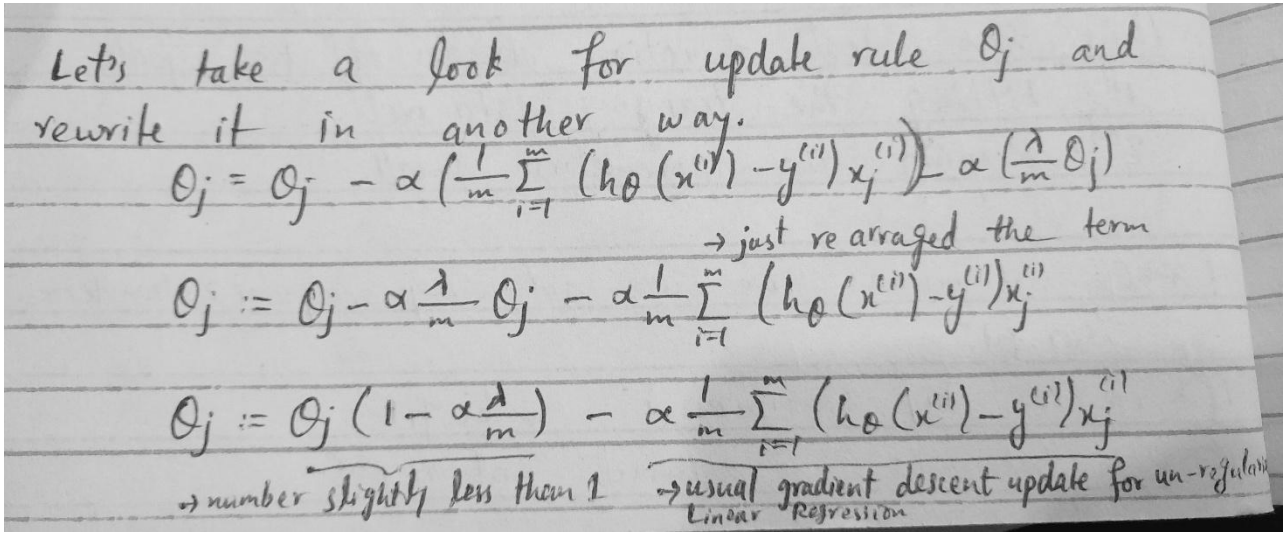


We don't have to penalize θ_0



Regularized linear regression

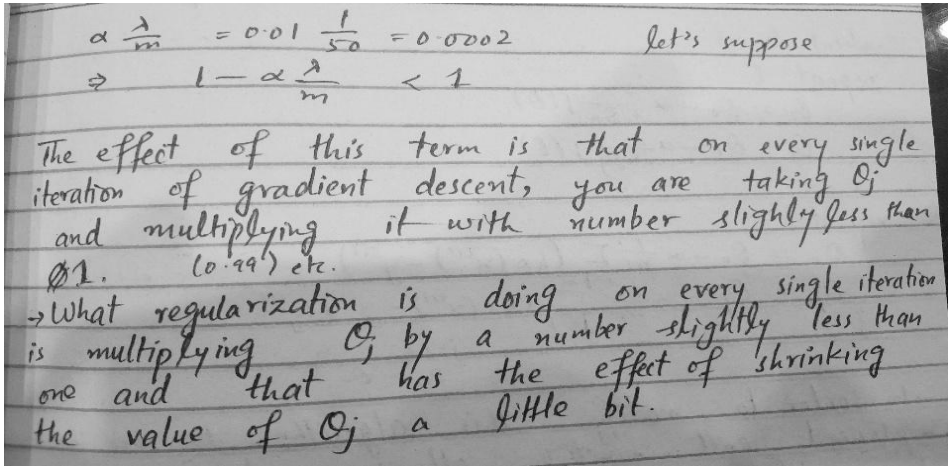
Gradient Descent:

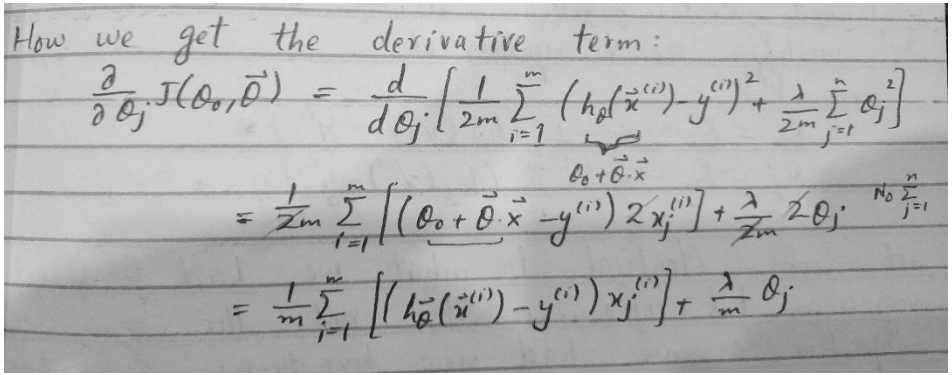


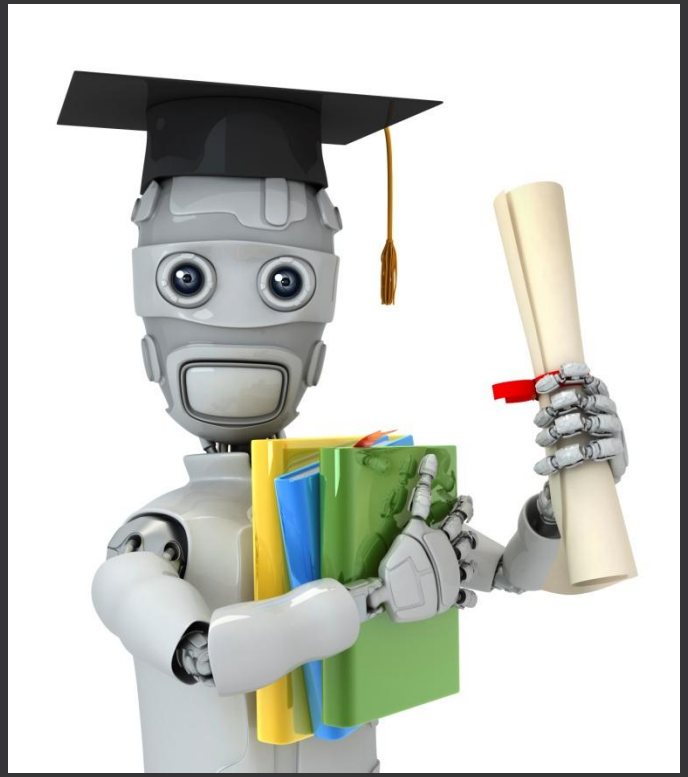
$$\theta_j := \theta_j(1 - \alpha\frac{\lambda}{m}) - \alpha\frac{1}{m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})x_j^{(i)}$$

Regularized linear regression

Gradient Descent:





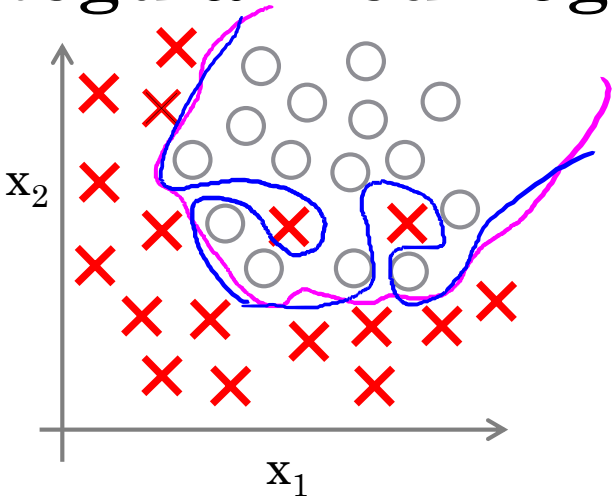


Machine Learning

Regularization

Regularized Logistic Regression

Regularized Logistic Regression.



$$h_\theta(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_1^2 + \theta_3 x_1^2 x_2 + \theta_4 x_1^2 x_2^2 + \theta_5 x_1^2 x_2^3 + \dots)$$

Cost function:

$$J(\theta) = -\left[\frac{1}{m}\sum_{i=1}^{m} y^{(i)} \log h_\theta(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_\theta(x^{(i)}))\right] + \frac{\lambda}{2m}\sum_{j=1}^{n} \theta_j^2$$

$\theta_1, \theta_2, \dots, \theta_n$

Gradient Descent

Repeat{

$$\theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x_0^{(i)}$$

$$\theta_j := \theta_j - \alpha \left[\frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x_j^{(i)} + \frac{\lambda}{m} \theta_j \right] \quad (j = \cancel{0}, 1, 2, 3, \ldots, n)$$

$\theta_1 \ldots \theta_n$

}

$\frac{\partial}{\partial \theta_j} J(\theta)$

$h_\theta(x) = \frac{1}{1 + e^{-\theta^T x}}$

Thank You